

ユニット 3・第3-4週

ファイルとファイルシステム

📅 第3-4週・日程は未定 </> 演習 TD2（解答 + 生成AIチャレンジ） ~ アニメーション 4 本

演習 TD2・解答付き

ファイル：fstat、FIFO、dup、ディレクトリ

Unix の「ファイル」はディスク上の文書以上のものです：端末、パイプ、ディレクトリ、ソケットのこともあります。この演習では、それらを記述し、開き、走査する呼び出しを扱います。

☑ 課題：実習レポート

この実習は短い LaTeX レポート（必要なことだけ）で提出します。扱う内容、テンプレート、例、Prism のガイドは「課題」タブにあります。生成 AI は申告すれば利用できます。

演習問題

- | | | | |
|---|--------------------|---|-------------------------|
| 1 | fstat で標準入力の種類を調べる | 4 | 自分の ls |
| 2 | 名前付きパイプ（FIFO）でチャット | 5 | さらに進んで：N クライアントの双方向チャット |
| 3 | 2 回の open と dup | | |

TD2 ・ 1

● 解答済み・テスト済み

fstat(2) で標準入力の種類を調べる

課題。fstat(STDIN_FILENO, ...) と st_mode に対する S_IS... マクロで、標準入力の種類を表示してください。端末、< /etc、実行ファイル自身、パイプで試しましょう（inode(7) を参照）。

解答



C tds/td2/filetype.c

```

/*
 * TD2 - exercise 1: type of the standard input with fstat(2).
 * ./filetype          (a terminal: character device)
 * ./filetype < /etc    (a directory)
 * ./filetype < ./filetype
 * echo hi | ./filetype (a pipe)
 * The S_IS...() macros test the st_mode field (see inode(7)).
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

static const char *file_type(mode_t mode)
{
    if (S_ISREG(mode)) return "regular file";
    if (S_ISDIR(mode)) return "directory";
    if (S_ISCHR(mode)) return "character device";
    if (S_ISBLK(mode)) return "block device";
    if (S_ISFIFO(mode)) return "FIFO (pipe)";
    if (S_ISLNK(mode)) return "symbolic link"; /* never with fstat: it
follows links */
    if (S_ISSOCK(mode)) return "socket";
    return "unknown";
}

int main(void)
{
    struct stat st;

    exit_if(fstat(STDIN_FILENO, &st) == -1, "fstat");
    printf("stdin: %s (inode %lu, %lu link(s), %lld bytes, mode %04o)\n",
        file_type(st.st_mode), (unsigned long)st.st_ino, (unsigned
long)st.st_nlink,
        (long long)st.st_size, (unsigned)(st.st_mode & 07777));
    return EXIT_SUCCESS;
}

```

実際の出力

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288 bytes, mode 0755)
```

実際の出力

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584 bytes, mode 0775)
```

実際の出力

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes, mode 0600)
```

プログラムは何も開きません：シェルが実行前にファイルを開き、ディスクリプタ 0 を渡しています。 `fstat` はそのディスクリプタの裏にある **inode** についてカーネルに問い合わせます。パイプにも inode があり（カーネル内部のファイルシステム `pipefs`）、サイズは 0：データはどのディスクにもありません。端末では種類はキャラクタデバイス（`/dev/pts/N`）です。 `fstat` では `S_ISLNK` が真になることはありません。ディスクリプタはすでにリンク先を指しているからです。リンクそのものを見るにはパスに対して `lstat` が必要です（演習問題 4）。

TD2 ・ 2

● 解答済み・テスト済み

名前付きパイプ（FIFO）でチャット

課題。FIFO はパイプを表す特殊ファイルです：あるプロセスが書いたものを別のプロセスが読み、データはディスクを通りません。(1) FIFO を書き込み用に開いて標準入力を送る `fifo_client` を書く。(2) `mkfifo` / `mknod` で FIFO を作り、読み込み用に開いて届いたものを表示する `fifo_server` を書く。(3) 複数のクライアントが同時にいるとどうなる？(4) バイナリファイル（画像）を送るには何を変える必要がある？(5) FIFO がネットワーク共有フォルダにあり、サーバーが別のマシンで動いていても機能する？

解答



C tds/td2/fifo_server.c

```

/*
 * TD2 - exercise 2: chat server over a named pipe (FIFO).
 * Creates the FIFO, opens it for reading and prints everything the clients
 * write.
 * ./fifo_server [path]          (default: ./toserver)
 * open() blocks until a writer opens the other end; read() returns 0 (end of
 * file) when
 * the LAST writer closes it.
 */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    /* Remove a FIFO left by a previous run; ENOENT (it did not exist) is
    fine. */
    exit_if(unlink(path) == -1 && errno != ENOENT, "unlink");

    /* 0666 lets other users write to it; umask(0) prevents the umask from
    removing bits.
    mknod(path, S_IFIFO | 0666, 0) is equivalent (the device number is
    unused). */
    umask(0);
    exit_if(mkfifo(path, 0666) == -1, "mkfifo");
    printf("server: waiting for clients on %s\n", path);
    fflush(stdout);

    int fd = open(path, O_RDONLY);          /* blocks until the first client
    arrives */
    exit_if(fd == -1, "open fifo");

    transfer(fd, STDOUT_FILENO);           /* until the last client leaves */

    exit_if(close(fd) == -1, "close");
    exit_if(unlink(path) == -1, "unlink");

```

```
    return EXIT_SUCCESS;
}
```

C tds/td2/fifo_client.c

```
/*
 * TD2 - exercise 2: chat client. Sends its standard input to the server
 * FIFO.
 * ./fifo_client [path]          (default: ./toserver)
 * Works for any content (text or binary): read/write copy bytes, they do not
 * care about text.
 * ./fifo_client < image.png     sends a whole file
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    int fd = open(path, O_WRONLY);          /* O_WRONLY: we only write */
    exit_if(fd == -1, "open fifo");

    transfer(STDIN_FILENO, fd);

    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' |
./fifo_client chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

実際の出力

```
$ ./fifo_server bin.fifo > received.bin & sleep 0.3; ./fifo_client bin.fifo <
photo.bin; wait; cmp photo.bin received.bin && echo 'binary file received
intact'
photo.bin received.bin differ: byte 1, line 1
[exit status 1]
```

1. **互いを待つ open。**読み込み用の `open` は誰かが書き込み用に開くまでブロックし、逆も同じです：サーバーは CPU を使わずに「クライアント待ち」の状態です。
2. **複数のクライアント。**全員が同じパイプに書き、サーバーは届いた順に混ざったメッセージを受け取ります。 `PIPE_BUF` バイト（Linux では 4096）以下の各 `write` は原子的なので、短いメッセージ同士が途中で切れることはありませんが、誰が送ったかはわかりません。最後のクライアントが去るとサーバーはファイル終端を見ます。
3. **バイナリファイル。**何も変える必要はありません： `read / write` はテキストではなくバイトをコピーします。テストでは 150 KB の実行ファイルを送り、 `cmp` で同一であることを確かめています。
4. **別のマシン。**機能しません：FIFO は 1 台のマシンのカーネルのオブジェクトです。ネットワーク共有フォルダで共有されるのは名前だけで、開くときに各カーネルがそれぞれのパイプを作ります。マシン同士の通信にはソケットが必要です（ユニット 8）。

⚠ 変更点

元の課題では比較のために ENSEIRB のサーバーへ SSH 接続することになっていましたが、ここでは問 5 でその環境に頼らずに同じ現象を扱います。 `transfer` 関数は `libutils` に移しました（ `EINTR` と部分書き込みのときの再試行付き）。

TD2 ・ 3

● 解答済み・テスト済み

2 回の `open` と `dup`

課題。 `donnees.txt` = `abcdefgh` とします：ファイルを開き、4 バイト読んで表示します。次に `alias_open` ではファイルをもう一度開いて新しいディスクリプタで 4 バイト読み、 `alias_dup` では `dup` でディスクリプタを複製して同じことをします。比べてみましょう。

解答



C tds/td2/alias_open.c

```

/*
 * TD2 - exercise 3: two open() of the same file = two independent offsets.
 * With donnees.txt = "abcdefgh" this prints "abcdabcd".
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define DATAFILE "donnees.txt"
#define CHUNK 4

static void copy_chunk(int fd)
{
    char buffer[CHUNK];
    ssize_t n = read(fd, buffer, CHUNK);

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
}

int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                                /* "abcd": offset of fd1 is now 4
*/

    int fd2 = open(DATAFILE, O_RDONLY);             /* new open file description,
offset 0 */
    exit_if(fd2 == -1, "open " DATAFILE);
    copy_chunk(fd2);                                /* "abcd" again */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}
    
```

C tds/td2/alias_dup.c

```
int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                /* "abcd": the shared offset is
now 4 */

    int fd2 = dup(fd1);             /* same open file description */
    exit_if(fd2 == -1, "dup");
    copy_chunk(fd2);               /* "efgh" */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./alias_open
abcdabcd
```

実際の出力

```
$ ./alias_dup
abcdefgh
```

カーネルには3つの層があります：プロセスのディスクリプタ表 → オープンファイル記述（オフセットとモードを持つ） → **inode**。 `open` のたびにオフセット0の新しい記述が作られるので、`abcd` が2回読めます。 `dup` は同じ記述を指す2つ目のディスクリプタを作るので、オフセットを共有して `efgh` が読めます。 `fork` でも同じで（子は共有された記述を引き継ぐ）、シェルの `2>&1` はこれを使っています。

アニメーション A3.2・ディスクリプタ表で見る →

opendir、readdir、lstat で作る自分の ls

課題。myls を書いてください：引数で与えたディレクトリ（なければ .）の内容を、種類を表す文字（-、d、l...）を前に付けて一覧表示します。発展：getopt(3) でオプション -l（権限、所有者、サイズ、日付）と -R（再帰）を加える。

解答

C tds/td2/myls.c

```
/*
 * TD2 - exercises 4 and 5.4: a small ls with opendir/readdir/closedir and
 * lstat.
 * ./mysls [-l] [-R] [dir...]      (no directory = ".")
 * -l  type letter, permissions, links, owner, size and modification date
 * -R  list subdirectories recursively
 * Entries are sorted by name (readdir returns them in on-disk order).
 */
#include <dirent.h>
#include <grp.h>
#include <limits.h>
#include <pwd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static int opt_long = 0;
static int opt_recursive = 0;

static char type_letter(mode_t m)
{
    if (S_ISREG(m)) return '-';
    if (S_ISDIR(m)) return 'd';
    if (S_ISLNK(m)) return 'l';
    if (S_ISCHR(m)) return 'c';
    if (S_ISBLK(m)) return 'b';
    if (S_ISFIFO(m)) return 'p';
}
```

```

    if (S_ISSOCK(m)) return 's';
    return '?';
}

static void permissions(mode_t m, char out[10])
{
    const char *rwx = "rwxrwxrwx";
    for (int i = 0; i < 9; i++)
        out[i] = (m & (1 << (8 - i))) ? rwx[i] : '-';
    out[9] = '\0';
}

static int by_name(const void *a, const void *b)
{
    return strcmp(*(char *const *)a, *(char *const *)b);
}

static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue
with the others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
    qsort(names, n, sizeof *names, by_name);

    if (show_header)

```

```

    printf("%s:\n", dir);
    for (size_t i = 0; i < n; i++) {
        char path[PATH_MAX];
        struct stat st;

        /* snprintf never overflows; the original sprintf into char[255]
        could. */
        if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
            (int)sizeof path) {
            fprintf(stderr, "%s/%s: path too long\n", dir, names[i]);
            continue;
        }
        if (lstat(path, &st) == -1) {      /* lstat: describes the link
        itself */
            perror(path);
            continue;
        }
        if (opt_long) {
            char perm[10], date[32];
            struct passwd *pw = getpwuid(st.st_uid);
            struct group *gr = getgrgid(st.st_gid);
            permissions(st.st_mode, perm);
            strftime(date, sizeof date, "%Y-%m-%d %H:%M",
localtime(&st.st_mtime));
            printf("%c%s %3lu %-8s %-8s %8lld %s %s",
type_letter(st.st_mode), perm,
                (unsigned long)st.st_nlink, pw ? pw->pw_name : "?", gr ?
gr->gr_name : "?",
                (long long)st.st_size, date, names[i]);
            if (S_ISLNK(st.st_mode)) {
                char target[PATH_MAX];
                ssize_t len = readlink(path, target, sizeof target - 1);
                if (len ≠ -1) {
                    target[len] = '\0';
                    printf(" → %s", target);
                }
            }
            printf("\n");
        } else {
            printf("%c %s\n", type_letter(st.st_mode), names[i]);
        }
    }

    if (opt_recursive) {
        for (size_t i = 0; i < n; i++) {

```

```

        char path[PATH_MAX];
        struct stat st;
        if (!strcmp(names[i], ".") || !strcmp(names[i], ".."))
            continue;
        if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
(int)sizeof path)
            continue;
        if (lstat(path, &st) == 0 && S_ISDIR(st.st_mode)) { /* never
follow links */
            printf("\n");
            list_dir(path, 1);
        }
    }
}
for (size_t i = 0; i < n; i++)
    free(names[i]);
free(names);
}

int main(int argc, char *argv[])
{
    int c;

    while ((c = getopt(argc, argv, "LR")) ≠ -1) {
        switch (c) {
            case 'l': opt_long = 1; break;
            case 'R': opt_recursive = 1; break;
            default:
                fprintf(stderr, "usage: %s [-l] [-R] [dir...]\n", argv[0]);
                return EXIT_FAILURE;
        }
    }

    int ndirs = argc - optind;
    if (ndirs == 0) {
        list_dir(".", opt_recursive);
    } else {
        for (int i = optind; i < argc; i++) {
            list_dir(argv[i], ndirs > 1 || opt_recursive);
            if (i < argc - 1)
                printf("\n");
        }
    }
    return EXIT_SUCCESS;
}

```

実際の出力

```
$ ./myls demo
d .
d ..
- a.txt
l link
p pipe
d sub
```

実際の出力

```
$ ./myls -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
```

- `readdir` が返すのは**名前**だけです（ディレクトリは「名前 → inode 番号」の表）。種類と権限を知るには、完全なパスに対して `lstat` で inode を問い合わせる必要があります。
- `stat` ではなく `lstat` : `stat` では `link` が通常のファイル（リンク先）として表示され、`-R` がリンクをたどって無限に回るおそれがあります。
- `.` と `..` はどのディレクトリにも実在するエントリです（だから空のディレクトリのリンク数は 2）。

⚠ 修正版

引き継いだ版は 255 バイトの配列に `sprintf` でパスを組み立てていたため、長い名前でスタックがあふれていました（gcc 15 はこれを検出し、`-Werror` ではビルドを拒否します）。現在は `snprintf` と `PATH_MAX` を使い、エントリを並べ替え、`-l` と `-R` を実装しています。

さらに進んで：N クライアントの双方向チャット

答え

- **なぜ FIFO を 1 つではなく 2 つ使う？** パイプは 1 本の流れです：両端が同じパイプを読み書きすると、自分のメッセージを自分で読んでしまうことがあります（バイトは先に `read` した者が消費する）。`toserver` と `toclient` なら、向きごとに専用の通路があります。
- **N クライアント。** サーバーへの共通の入力 FIFO を 1 つ、クライアントごとに出力 FIFO を 1 つ用意し、サーバーが各メッセージをすべての出力に転送します。出力 FIFO が 1 つだけではうまくいきません：クライアント同士がメッセージを「奪い合う」からです。
- **自分のメッセージを見ないようにする。** 各メッセージの先頭にクライアントの識別子（たとえば `pid`）を付け、サーバーは送り主には転送しません。これがこの演習の AI チャレンジです。

チェックリスト

生成AIチャレンジ・自分のメッセージが返ってこないNクライアントのチャット



目標。 TD2 の「さらに進んで」の問いを解きましょう：FIFO を使い、どのクライアントにも自分のメッセージが表示されない N クライアントの双方向チャット。

最初のプロンプト。 好きなツール（Claude、ChatGPT、Gemini、Copilot、Codex...）で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

FIFO を 1 つ使う C のチャットサーバーとクライアントがあります（クライアントが書き、サーバーが表示する）[fifo_server.c と fifo_client.c を貼る]。互いのメッセージが見える N クライアントに拡張したいです：(1) 共通の入力 FIFO 1 つとクライアントごとの出力 FIFO を使う設計を提案し、全員が 1 つの出力 FIFO から読むと何が問題かを説明する、(2) 送り主に表示しないよう、各メッセージにクライアントの識別子を付ける、(3) FIFO のブロッキングする open で何が起こるか、開くときのデッドロックをどう避けるかを説明する。コードは一歩ずつ示し、各ステップの間で私の確認を待ってください。

結果の確かめ方

- 3 つの端末で、それぞれが書いたものが他の 2 つに表示され、自分には表示されない。
- クライアントを 1 つ閉じてても他はブロックも終了もしない。
- 同じ FIFO を読む 2 つの読み手がなぜメッセージを「奪い合う」のかを説明できる。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。